

Paradigmas de programação e reutilização: Orientação a Objetos para reuso

QXD0068 - Reuso de Software

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

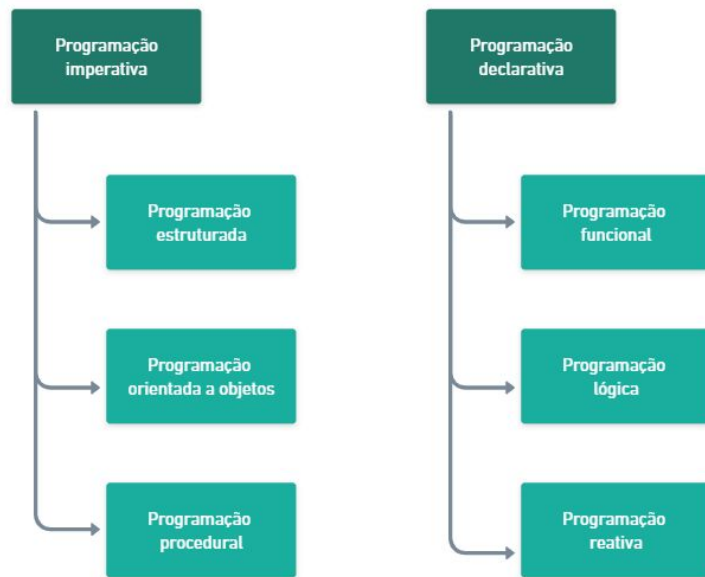
- Motivação
- O que são Paradigmas de Programação?
- Exemplos de paradigmas principais
- Paradigmas e Reutilização de Software
- Comparação entre Paradigmas
- Exemplos Práticos de Reutilização
- Abstração na OO
- Exemplo (Python)
- Parametrização na OO
- Parametrização por Tipos (Generics)
- Parametrização por Comportamento
- Relação com Padrões de Projeto

Motivação

- A programação evoluiu em estilos (paradigmas) que influenciam como pensamos e estruturamos soluções.
- Paradigmas não são apenas “formas diferentes de programar”, mas modos de raciocinar sobre problemas.
- Entender paradigmas é essencial para escolher ferramentas adequadas e potencializar a reutilização de software.
- Reutilização se beneficia de abstrações e boas práticas que cada paradigma pode trazer.

O que são Paradigmas de Programação?

- **Definição:** Modelo conceitual que orienta a forma de resolver problemas por meio de linguagens e técnicas.
- **Impacto:** Influenciam como projetamos, estruturamos e reutilizamos código.



Exemplos de paradigmas principais

- **Imperativo / Procedural**
 - Passos sequenciais, foco no como fazer.
 - Linguagens: C, Pascal.
 - Reutilização: funções/procedimentos.
- **Orientado a Objetos (OO)**
 - Modela o mundo com classes, objetos, herança, polimorfismo.
 - Linguagens: Java, C++, Python.
 - Reutilização: bibliotecas, frameworks, padrões de projeto.

Exemplos de paradigmas principais

- **Funcional**
 - Funções puras, imutabilidade, ausência de efeitos colaterais
 - Linguagens: Haskell, Elixir, Scala; suporte em Python/JavaScript.
 - Reutilização: composição de funções, funções de ordem superior.
- **Lógico / Declarativo**
 - Especifica o que deve ser feito, não como.
 - Linguagens: Prolog, SQL.
 - Reutilização: consultas e regras reutilizáveis.
- **Concorrente / Paralelo**
 - Estruturas para execução simultânea.
 - Linguagens: Go, Erlang, Java (threads).
 - Reutilização: padrões de concorrência, bibliotecas reativas.

Paradigmas e Reutilização de Software

- **Imperativo/Procedural:**
 - Reuso por procedimentos e módulos.
 - Limitação: acoplamento forte entre dados e funções.
- **OO:**
 - Forte suporte à reutilização com herança, interfaces, composição e design patterns.
 - Exemplo: classes de utilidades, bibliotecas de UI.

Paradigmas e Reutilização de Software

- **Funcional:**
 - Reuso com funções puras → fáceis de testar, compor e transportar entre projetos.
 - Exemplo: funções map/reduce reutilizáveis em Big Data.
- **Lógico/Declarativo:**
 - Reuso de consultas, regras e constraints em diferentes cenários.
 - Exemplo: SQL para diferentes bancos sem reescrever lógica.
- **Concorrente/Paralelo:**
 - Reuso de modelos de atores, streams, workers em vários contextos.
 - Exemplo: bibliotecas de processamento paralelo.

Comparação entre Paradigmas

Paradigma	Abstração central	Reutilização típica	Exemplo
Imperativo	Procedimentos	Funções/módulos	Função sort t em C
OO	Objetos/classes	Herança, padrões	Biblioteca Swing/Java
Funcional	Funções puras	Composição, map/reduce	Funções em Python (map, filter)
Lógico	Regras e fatos	Consultas reutilizáveis	Queries SQL
Concorrente	Processos/threads	Modelos de atores	Go routines, Erlang OTP

Exemplos Práticos de Reutilização

- **Imperativo:** printf, scanf (C) usados em diversos programas.
- **OO:** Classe ArrayList do Java → usada em diferentes sistemas.
- **Funcional:** map() no Python → abstração genérica de iteração.
- **Declarativo:** SQL JOIN reaproveitado em múltiplos contextos de banco de dados.
- **Concorrente:** async/await em JavaScript → padrão reutilizável para chamadas assíncronas.

Reflexão

- Nenhum paradigma é “melhor” em absoluto → cada um é mais adequado a certos problemas.
- A tendência atual: hibridismo (ex.: Python mistura procedural, OO e funcional; Scala combina OO e funcional).
- Reutilização é maximizada quando o paradigma favorece abstrações claras e baixo acoplamento.
- Como engenheiros de software, precisamos ser multilíngues em paradigmas, escolhendo o certo para cada domínio e promovendo reuso.

Reflexão

- Nenhum paradigma é “melhor” em absoluto → cada um é mais adequado a certos problemas.
- A tendência atual: hibridismo (ex.: Python mistura procedural, OO e funcional; Scala combina OO e funcional).
- Reutilização é maximizada quando o paradigma favorece abstrações claras e baixo acoplamento.
- Como engenheiros de software, precisamos ser multilíngues em paradigmas, escolhendo o certo para cada domínio e promovendo reuso.

Orientação a Objetos para reuso: abstração e parametrização

- OO é o paradigma que mais explicitamente incentiva reutilização de software.
- Através de abstração e parametrização, podemos construir componentes que se adaptam a diversos cenários sem reescrever código.
- Essa capacidade está diretamente ligada a conceitos como baixo acoplamento, alta coesão, generalização e flexibilidade.

Abstração na OO

- **Definição**

- Abstração é o processo de isolar características essenciais de um objeto, ignorando detalhes irrelevantes para o contexto.
- É a base para definir interfaces, classes abstratas e modelos conceituais.

- **Características**

- Esconde a complexidade interna.
- Permite contratos (interfaces) que várias implementações podem seguir.
- Facilita a substituição de componentes sem afetar o resto do sistema (polimorfismo).

Exemplo (Python)

```
from abc import ABC, abstractmethod

# Abstração: Interface de um repositório
class Repositorio(ABC):
    @abstractmethod
    def salvar(self, id: int, entidade):
        pass

    @abstractmethod
    def buscar_por_id(self, id: int):
        pass
```

Exemplo (Python)

```
# Implementação 1: Repositório em memória

class RepositorioMemoria(Repositorio):
    def __init__(self):
        self.dados = {}

    def salvar(self, id: int, entidade):
        self.dados[id] = entidade
        print(f"[Memória] Entidade salva com id={id}")

    def buscar_por_id(self, id: int):
        return self.dados.get(id, None)
```

Exemplo (Python)

```
# Implementação 2: Repositório em banco de dados (simulado)

class RepositorioBD(Repositorio):
    def salvar(self, id: int, entidade):
        # Aqui poderia entrar código SQL real
        print(f"[BD] Inserindo entidade com id={id} no banco...")

    def buscar_por_id(self, id: int):
        # Aqui poderia haver um SELECT no banco
        print(f"[BD] Buscando entidade com id={id} no banco...")
        return None
```

Exemplo (Python)

```
# Código cliente

def cliente(repositorio: Repositorio):
    repositorio.salvar(1, {"nome": "Maria"})
    entidade = repositorio.buscar_por_id(1)
    print("Resultado da busca:", entidade)

if __name__ == "__main__":
    print("Usando Repositório em Memória:")
    cliente(RepositorioMemoria())

    print("\nUsando Repositório em Banco de Dados:")
    cliente(RepositorioBD())
```

Saída esperada (simulação):

Usando Repositório em Memória:

[Memória] Entidade salva com id=1

Resultado da busca: {'nome': 'Maria'}

Usando Repositório em Banco de Dados:

[BD] Inserindo entidade com id=1 no banco...

[BD] Buscando entidade com id=1 no banco...

Resultado da busca: None

Parametrização na OO

- **Definição**

- Parametrização significa tornar generalizável uma classe/método, recebendo parâmetros de tipo ou comportamento.
- Garante reutilização sem duplicação de código.

- **Tipos**

- Parametrização por Tipos (Generics)
- Parametrização por Comportamento (Strategy, Callback, Função como parâmetro)

Parametrização por Tipos (Generics)

- Define classes ou métodos que funcionam para múltiplos tipos.
- Exemplo: `List<T>` no Java, `ArrayList<T>`.
- Benefício: reuso do mesmo algoritmo com diferentes tipos de dados.

Parametrização por Comportamento

- (Strategy, Callback, Função como parâmetro)
- Permite passar o comportamento a ser executado.
- Evita código duplicado para pequenas variações de lógica.

Exemplo (Java Generics)

```
// Classe genérica: funciona para qualquer tipo
public class Caixa<T> {
    private T valor;

    public void setValor(T valor) { this.valor = valor; }
    public T getValor() { return valor; }
}
```

```
// Uso:
Caixa<String> caixaTexto = new Caixa<>();
caixaTexto.setValor("Olá");
System.out.println(caixaTexto.getValor());

Caixa<Integer> caixaNumero = new Caixa<>();
caixaNumero.setValor(42);
System.out.println(caixaNumero.getValor());
```

Uma única implementação é **reutilizada** para diferentes tipos, sem precisar criar `CaixaString`, `CaixaInteger`, etc.

Exemplo (Python)

```
from typing import Generic, TypeVar

T = TypeVar("T")

# Classe genérica
class Caixa(Generic[T]):
    def __init__(self):
        self.valor: T | None = None

    def set_valor(self, valor: T) -> None:
        self.valor = valor

    def get_valor(self) -> T:
        return self.valor

# Uso
caixa_texto = Caixa[str]()
caixa_texto.set_valor("Olá")
print(caixa_texto.get_valor()) # Olá
```

Em Python puro, já existe “reuso automático” porque os tipos são dinâmicos.

O uso de `Generic[T]` é apenas para anotações de tipo e para ferramentas como mypy ou IDEs detectarem inconsistências, deixando o código mais robusto.

Exemplo (Strategy Pattern - Parametrização por Comportamento)

```
from abc import ABC, abstractmethod

# Estratégia de cálculo de imposto
class EstrategiaImposto(ABC):
    @abstractmethod
    def calcular(self, valor: float) -> float: ...

class ImpostoBrasil(EstrategiaImposto):
    def calcular(self, valor): return valor * 0.15

class ImpostoEUA(EstrategiaImposto):
    def calcular(self, valor): return valor * 0.10

class CalculadoraImposto:
    def __init__(self, estrategia: EstrategiaImposto):
        self.estrategia = estrategia
    def calcular(self, valor): return self.estrategia.calcular(valor)
```

Reuso máximo: a lógica da calculadora é fixa, mas o comportamento é parametrizado.

Benefícios do Reuso com Abstração e Parametrização

- Generalização sem perda de expressividade.
- Menos duplicação → reduz custo de manutenção.
- Maior flexibilidade → mudanças afetam apenas pontos específicos.
- Extensibilidade → sistemas podem crescer com novas variantes sem alterar código existente.

Relação com Padrões de Projeto

- Factory Method / Abstract Factory → abstração de criação.
- Strategy → parametrização de comportamento.
- Template Method → abstração com pontos de variação.
- Decorator → parametrização incremental de comportamento.

Reflexão

- OO para reuso não é apenas herança.
- O verdadeiro poder vem de abstrações bem definidas e parametrizações inteligentes, que permitem código mais flexível, genérico e duradouro.
- Quanto melhor o design, maior o potencial de reuso em diferentes domínios e projetos.

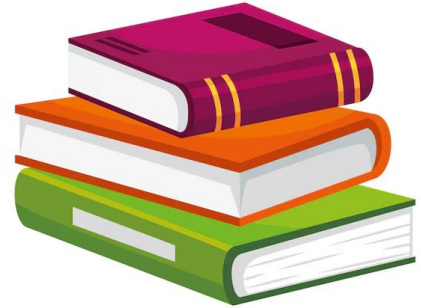
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

